CMDO 2025 - Sports Tournament Scheduling (STS) problem

Luca Anzaldi luca.anzaldi@studio.unibo.it
Martino Michelotti martino.michelotti@studio.unibo.it
Alex-Cristian Cozma alexcristian.cozma@studio.unibo.it

May 31, 2026

1 Introduction

This report addresses the solution of the well-known *Sports Tournament Scheduling Problem (STS[2])*. The focus is on presenting different modeling approaches and optimization techniques to handle the scheduling constraints and objectives. To avoid repetition, later sections will refer back to this introduction whenever these shared components are required.

Instance parameters Let $T = n$ be the number of teams (even), $W = T - 1$ the number of weeks, and $P = T/2$ the number of periods (parallel timeslots) per week. Each solution is modeled referring to these three main variables.

For the sake of brevity, whenever w, p and t (or eventually t_1, t_2) are used in formulas and the domains are omitted, they have to be implicit meant to be: $w \in \{1, \ldots, W\}$, $p \in \{1, \ldots, P\}$ and $t \in \{1, \ldots, T\}$.

General constraints The classical formulation of the STS problem requires that:

1. every team plays with every other team exactly once;
2. every team plays exactly once per week;
3. every team plays at most twice in the same period over the whole tournament.

Objective function The main optimization criterion is to achieve a balanced schedule, in which every team plays approximately the same number of home and away games. The objective is to minimize the overall imbalance:

$$\sum_{t=1}^{T} \Big| \, \#\text{home\_games}(t) - \#\text{away\_games}(t) \, \Big|$$

2 SAT Model

The SAT solution was made using two different modeling approaches. Both are included in the report.

Model 1 was implemented using several alternative SAT encodings for the cardinality constraints introduced above. In particular, we considered the **Bitwise** encoding, the **Heule** encoding, the **Pairwise** encoding, and the **Sequential** encoding. Each encoding provides a different trade-off between the number of variables, the number of clauses, and the strength of constraint propagation.

Model 2 extends the formulation of Model 1 by adding further constraints and auxiliary decision variables. In contrast to Model 1, the implementation adopts a single SAT encoding approach (e.g., pairwise-style cardinality constraints) and does not perform a systematic comparison across alternative encodings.

Moreover, the implementation can optionally enable a **precomputing** phase described in paragraph 2.3.2.

2.1 Decision variables

2.1.1 Model - 1

The use of multiple encodings within the same model allows for a systematic comparison of their impact on solver performance, while keeping the underlying problem formulation fixed.

Booleans

- $X_{t,h,p,w} \in \{0,1\}$ meaning team t is scheduled with role h in slot (p,w).

2.1.2 Model - 2

Booleans

- $M_{t_1,t_2,w} \in \{0,1\}$ meaning teams t_1 and t_2 play against each other in week w.
- $HOME_{t,w} \in \{0,1\}$ meaning team t plays at home in week w (away otherwise).
- $P_{t,p,w} \in \{0,1\}$ meaning team t is assigned to period p in week w.

Remarks. Model–2 introduces explicit pair variables $M_{t_1,t_2,w}$ to capture which two teams meet in a given week, home assignment variables $HOME_{t,w}$ to distinguish roles, and period variables $P_{t,p,w}$ to ensure consistency of scheduling.

2.2 Objective function

2.2.1 Model - 1

We minimize total home/away imbalance across teams:

$$\sum_{t=1}^{T} \left| \sum_{p=1}^{P} \sum_{w=1}^{W} \left(X_{t,0,p,w} - X_{t,1,p,w} \right) \right|.$$

In the implementation with `Z3`, the optimization is performed by *iterative (binary) search* on a pseudo-Boolean bound z. At each step we introduce a constraint of the form $\sum_t |\cdot| \leq z$, encoded using cardinality constraints, and solve for feasibility. If satisfiable, the bound is tightened; otherwise the process stops at the last feasible value.

2.2.2 Model - 2

As in Model–1, the optimization criterion is to balance the number of home and away games for each team. Formally:

$$\sum_{t=1}^{T} \left| \sum_{w=1}^{W} HOME_{t,w} - \left(W - \sum_{w=1}^{W} HOME_{t,w} \right) \right|.$$

2.3 Constraints

2.3.1 Model - 1

Main constraints.

(C1) **Every pair of teams plays at most once:**

$$\forall\, t_1, t_2 \ \text{ with } \ t_1 \neq t_2 \quad \sum_{p=1}^{P} \sum_{w=1}^{W} \Big(\bigvee_{h \in \{0,1\}} X_{t_1,h,p,w} \wedge \bigvee_{h \in \{0,1\}} X_{t_2,h,p,w} \Big) \ \leq 1.$$

(C2) **Each team plays exactly once per week:**

$$\forall\, t, w \ \sum_{h \in \{0,1\}} \sum_{p=1}^{P} X_{t,h,p,w} \ = \ 1.$$

(C3) **Each team appears at most twice in the same period over the tournament:**

$$\forall\, t, p \ \sum_{h \in \{0,1\}} \sum_{w=1}^{W} X_{t,h,p,w} \ \leq \ 2.$$

Implied constraints.

(C4) **Each game slot has exactly one home and one away team:**

$$\forall\, p, w \quad \sum_{t=1}^{T} X_{t,1,p,w} = 1 \quad \text{and} \quad \sum_{t=1}^{T} X_{t,0,p,w} = 1.$$

2.3.2 Model - 2

Main constraints.

(C1) **Each team plays with every other team only once;**

$$\forall\, t_1 < t_2 \qquad \sum_{w=1}^{W} M_{t_1,t_2,w} \;=\; 1.$$

(C2) **Each team plays exactly once per week**

$$\forall\, t,\, w \qquad \sum_{p=1}^{P} P_{t,p,w} \;=\; 1.$$

(C3) **Each team plays at most twice in the same period**

$$\forall\, t,\, p \qquad \sum_{w=1}^{W} P_{t,p,w} \;\leq\; 2.$$

(C4) **Each slot (p, w) hosts exactly two teams.**

$$\forall\, p,\, w \qquad \sum_{t=1}^{T} P_{t,p,w} \;=\; 2.$$

Implied constraints.

(C5) **Home/away consistency when two teams meet in week w.**

$$\forall\, w,\, t_1 < t_2 \qquad M_{t_1,t_2,w} \;\Rightarrow\; \big(HOME_{t_1,w} \oplus HOME_{t_2,w}\big).$$

(C6) **Link between pair variables and period assignments.**

$$\forall\, w,\, t_1 < t_2 \qquad M_{t_1,t_2,w} \iff \bigvee_{p=1}^{P} \big(P_{t_1,p,w} \wedge P_{t_2,p,w}\big),$$

Precomputing tecnique.

(P1) **Precomputed round-robin pruning.** Let $\mathcal{R}_w \subseteq \{\{t_1, t_2\} \mid t_1 < t_2\}$ be the fixed set of pairs for week w produced by the round-robin generator. Then:

$$\forall\, w,\, t_1 < t_2 \qquad M_{t_1,t_2,w} = \begin{cases} 1, & \text{if } \{t_1, t_2\} \in \mathcal{R}_w, \\ 0, & \text{otherwise.} \end{cases}$$

2.4 Validation

Experimental Setup

All experiments were conducted using the `Z3` solver for SAT and optimization ([3]). The implementation is provided in Python, and the solver is invoked through the **Z3 API**.

Reproducibility instructions, including installation steps and execution commands, are provided in the accompanying `README.md` file.

Experimental results

The **following tables reports** the results of the SAT experiments. Each row corresponds to a different value of n. On the tables are reported tthe value of the objective value obtained using different approach and in brackets the computation time for the best solution found. If the instance is solved to optimality is in bold. In addition if the instance is proved to be unsatisfiable, it is indicate as `UNSAT` and if no answer is obtained within the time limit of 300s are indicate it with `N/A`. Each modeling approach have is own table (Tab. 1 and Tab. 2).

The **precomputation time** required to generate the round-robin pairs is not reported in the tables, as it is negligible: it remains below one second even for $n \approx 50$, and therefore has no practical impact on the overall computational performance.

n	Bitwise	Heule	Pairwise	Sequential
2	**2** (0s)	**2** (0s)	**2** (0s)	**2** (0s)
4	`UNSAT` (0s)	`UNSAT` (0s)	`UNSAT` (0s)	`UNSAT` (0s)
6	**6** (0s)	**6** (0s)	**6** (0s)	**6** (0s)
8	14 (1s)	24 (1s)	18 (1s)	24 (0s)
10	26 (6s)	36 (4s)	18 (7s)	20 (6s)
12	30 (87s)	44 (13s)	36 (40s)	30 (151s)
14	`N/A`	`N/A`	`N/A`	`N/A`
16	`N/A`	`N/A`	`N/A`	`N/A`

Table 1: Experimental results of Model 1.

n	Standard	Precomputing
2	**2** (–)	**2** (0s)
4	`UNSAT` (–)	`UNSAT` (0s)
6	**6** (–)	**6** (0s)
8	**8** (–)	**8** (0s)
10	**10** (–)	**10** (0s)
12	**12** (3s)	**12** (1s)
14	**14** (83s)	**14** (6s)
16	46 (244s)	**16** (10s)
18	`N/A`	44 (53s)
20	`N/A`	200 (272s)
22	`N/A`	`N/A`

Table 2: Experimental results of Model 2.

3 MIP Model

In this section we will discuss how we implemented a Mixed Integer Linear Program model able to solve the given Tournament Scheduling problem.

The main obstacle in the implementation of linear programs is the fact that the objective function and every constraint have to be linear in our variables;

therefore we transformed logic propositions and non-linear functions with linear inequalities, introducing, if needed, some auxiliary variable as for the implementation of the objective function.

3.1 Decision variables

Before defining the variables, we compute the list L of the possible match-ups, without caring about the order of the (i, j) team tuples (we impose $i < j$ for unicity). If $n = T$ is the number of teams, we have $|L| = \binom{n}{2} = \frac{n!}{2!(n-2)!}$, i.e. $\binom{n}{2}$ games to be played.

Our decision variables are then two 3-dimensional tensors of **boolean** values $Y_{w,p,(i,j)}$ and $H_{w,p,(i,j)}$, where

- $Y_{w,p,(i,j)} = 1 \iff$ teams i and j are playing one against the other in week w in period p;
- $H_{w,p,(i,j)} = 1 \iff$ team i (the team with lowest index since $\forall (i, j) \in L$, $i < j$) is playing home in week w in period p.

Clearly these variables have to be linked one with the other, and that's why we will introduce some variable-linking constraint to our model.

3.2 Objective function

The objective function is used only in the optimization version of the MIP model. Its purpose is to balance the number of home and away games played by each team.

For each team t, let h_t be the number of home games played by team t. Since each team plays exactly one match per week and there are $W = n - 1$ weeks, the number of away games is:

$$a_t = W - h_t.$$

The home/away imbalance of team t is therefore: $|h_t - a_t|$.

Since absolute values cannot be used directly in a linear objective function, we introduce an auxiliary integer variable b_t for each team t, representing the imbalance of that team. We impose the following linear constraints:

$$b_t \geq h_t - a_t\,, \qquad \text{and} \qquad b_t \geq a_t - h_t \qquad \forall t.$$

Since the objective minimizes the sum of the imbalance variables, each b_t will take the smallest value satisfying both inequalities, and therefore:

$$b_t = |h_t - a_t|.$$

The objective function of the optimization version is then:

$$\min \sum_{t=1}^{n} b_t.$$

In the decision version, no real objective function is optimized. A dummy constant objective is used only to run the feasibility model through the MIP solver.

3.3 Constraints

Variable linking constraint As already mentioned, we have to link the variables and this has to be done by adding linear inequalities to our linear program.

The only constraint we have to add is the one that ensures that $H_{w,p,(i,j)} \neq 1$ if teams i and j do not play against in week w in period p, i.e.

$$Y_{w,p,(i,j)} = 0 \implies H_{w,p,(i,j)} \neq 1.$$

This constraint can be translated in a linear inequality by imposing

$$H_{w,p,(i,j)} \leq Y_{w,p,(i,j)}.$$

Main constraints **(C1) Each pair of teams plays exactly once:**

$$\forall (i,j) \in L \qquad \sum_{w=1}^{W} \sum_{p=1}^{P} Y_{w,p,(i,j)} = 1.$$

(C2) Each team plays exactly once per week:

$$\forall t, w \qquad \sum_{p=1}^{P} \sum_{\substack{(i,j)\in L \\ i=t \vee j=t}} Y_{w,p,(i,j)} = 1.$$

(C3) Each team plays at most twice in the same period:

$$\forall t, p \qquad \sum_{w=1}^{W} \sum_{\substack{(i,j)\in L \\ i=t \vee j=t}} Y_{w,p,(i,j)} \leq 2.$$

(C4) Each slot (w, p) hosts exactly one game:

$$\forall w, p \qquad \sum_{(i,j)\in L} Y_{w,p,(i,j)} = 1.$$

We don't need to explicitly impose that exactly two teams play in each slot, since the possible games have already been represented as team pairs in L.

Symmetry breaking constraints

(C5) We fix each match in the first week by imposing:

$$\forall p = 1, \ldots, P \qquad Y_{1,p,(2p-1,\,2p)} = 1 \quad \text{and} \quad H_{1,p,(2p-1,\,2p)} = 1$$

(C6) Since the weeks are interchangeable in our formulation (constraints and objective do not depend on the week index), we can break the week-permutation symmetry by fixing the opponent of team 1 in each week, without fixing the period.

$$\forall w = 1, \ldots, W \qquad \sum_{p=1}^{P} Y_{w,p,(1,\,w+1)} = 1,$$

i.e., in week w team 1 plays against team $w+1$ in some period p.

(C7) In addition we break symmetry by fixing the main diagonal of the schedule matrix: in each diagonal slot $(w, p) = (p, p)$ (for the first P weeks), team 1 must be scheduled, without fixing the opponent. Formally:

$$\forall p = 1, \ldots, P \qquad \sum_{j=2}^{T} Y_{p,p,(1,j)} = 1.$$

3.4 Validation

Experimental setup. All experiments for the MIP formulation were conducted using the CBC solver through the `python-mip` library [4]. The implementation is provided in Python, and the solver is invoked through the `python-mip` API.

For each instance, two versions of the model were executed:

- `mip_cbc_decision`, corresponding to the decision version of the problem, where the goal is to find any feasible schedule;
- `mip_cbc_optimization`, corresponding to the optimization version, where the objective is to minimize the total home/away imbalance.

The interpretation of the reported results and the meaning of the table entries follow the same convention described in Section 2.4. For the decision column we just reported if we got a solution and the time it took to reach it.

3.5 Experimental results.

n	Decision	Optimization
2	Yes (0s)	**2** (0s)
4	`UNSAT`	`UNSAT`
6	Yes (0s)	**6** (0s)
8	Yes (0s)	**8** (0s)
10	Yes (7s)	**10** (12s)
12	Yes (89s)	**12** (283s)
14	No	`N/A`

Table 3: CBC results for the MIP model.

4 CP Model

In this last section we will discuss how we implemented the Constraint programming model.

With respect to the other two implementations, constraint programming admits a higher range of modeling possibilities and we tried to use them to model less sparse arrays, by avoiding the heavy use of boolean variables as we did in the other models.

4.1 Decision variables

Our decision variables are 3 different arrays:

- $P_{t,w} \in \{1, \ldots, P\}$ where

$$P_{t,w} = p \iff \text{team } t \text{ plays in period } p \text{ in week } w$$

- Boolean array $H_{t,w}$ where

$$H_{t,w} = \texttt{true} \iff \text{team } t \text{ plays home in week } w;$$

- $OPP_{t_1,w} \in \{1, \ldots, T\}$ where

$$OPP_{t_1,w} = t_2 \iff \text{team } t_1 \text{ plays against team } t_2 \text{ in week } w.$$

4.2 Objective function

For the objective function, we opted for a proxy of the one defined in the introduction. As discussed in section **??**, we used the `max` function instead of `abs` and the Home/Away imbalance was parametrized using just home games; but in this case, instead of summing up the imbalance for each team, we decided to look for the maximum value of imbalance among all teams and this turned out to be a good choice for performance improvement. The objective function in this section will be

$$\max_{t \in \{1,\ldots,n\}} \Big| \, \#\text{home\_games}(t) - \#\text{away\_games}(t) \, \Big|.$$

Such a function can be used instead of the original one since we are looking to the optimal value: n for the original formulation, 1 for the *max* objective function. Knowing a hypothetic value k_{max} of the *max* objective function, defines an upper bound on the values k_{OG} of the original objective function, in fact

$$k_{OG} \leq n \, k_{max} \qquad \forall \, n, k_{max}$$

and in case of optimality we clearly get

$$k_{max} = 1 \implies k_{OG} \leq n \implies k_{OG} = n.$$

4.3 Constraints

Main constraints.

(C1) **Every pair of teams plays at most once.** For this constraint we used the global constraint `alldifferent` in the following way:

$$\forall\, t \qquad \texttt{alldifferent}(\,\{\, OPP_{t,w} \,|\, w = 1, \ldots, n-1 \,\}\,).$$

(C2) **Each team plays exactly once per week.**

This constraint has not to be defined in this model because the array of variables $P_{t,w}$ admits just one value for every team in every week.

(C3) **Each team appears at most twice in the same period over the tournament.**

$$\forall t, p \qquad \big|\{w \,|P_{t,w} = p\}\big| \leq 2$$

Implied constraints. These constraints are crucial for the actual definition of the problem and the correct use of the decision variables

(C4) **No team plays against itself:**

$$\forall t, w \qquad OPP_{t,w} \neq t$$

(C5) **Matches are symmetric:**

$$\forall t, w \qquad OPP_{OPP_{t,w},w} = t$$

(C6) Since the weeks are interchangeable in our formulation (constraints and objective do not depend on the week index), we can break the week-permutation symmetry by fixing the opponent of team 1 in each week, without fixing the period.

$$\forall w = 1, \ldots, W \qquad \sum_{p=1}^{P} Y_{w,p,(1,\,w+1)} = 1,$$

i.e., in week w team 1 plays against team $w+1$ in some period p.

(C7) In addition we break symmetry by fixing the main diagonal of the schedule matrix: in each diagonal slot $(w, p) = (p, p)$ (for the first P weeks), team 1 must be scheduled, without fixing the opponent. Formally:

$$\forall p = 1, \ldots, P \qquad \sum_{j=2}^{T} Y_{p,p,(1,j)} = 1.$$

Symmetry breaking The problem has different symmetries and we tried to reduce the search space as much as possible, but maintaining some flexibility. Some of the subsequent constraints seem redundant, but we experimentally found out they were improving performance by helping propagation. The constraints are imposed especially on team 1 and week 1 for simplicity, but they could've been imposed on every other week and team (singularly) because of the week and team symmetries.

(C9) We fix each match in the first week by imposing three combined constraints:

- fixing match-ups:

$$\forall p \qquad OPP_{2p-1,1} = 2p \ \wedge \ OPP_{2p,1} = 2p - 1 \ ;$$

- fixing periods:

$$\forall p \qquad P_{2p-1,1} = p \ \wedge \ P_{2p,1} \ ;$$

- fixing home/away teams:

$$\forall p \qquad H_{2p-1,1} = \texttt{true} \ \wedge \ H_{2p,1} = \texttt{false} \ .$$

(C10) We impose an increasing order of the opponents for team 1 with the following constraint:

$$\forall w = 1, \ldots, n-2 \qquad OPP_{1,w} < OPP_{1,w+1}.$$

This constraint helps, along with (C9), to break the team symmetry of the problem by defining a total order on the teams' labels.

4.4 Validation

Experimental design. All experiments for the constraint programming model were done using MiniZinc IDE and then implemented on the docker container. The model is runnable using Gecode solver, but with poor performance; we had some slight improvement when used with Optimization level O2 (two pass compilation). The model was instead optimized for the Chuffed solver (v 0.13.2), since we noticed it's efficiency already from the early stages of modeling. For reproducibility we fixed the random seed 42.

The experiment were ran on a *Windows 11* OS and the hardware platform was a laptop equipped with a *AMD RYZEN 7 5800H* processor. No GPU acceleration was employed for CP solvers, which ran entirely on the CPU.

Experimental results. The interpretation of the reported results and the meaning of the table entries follow a similar convention to the one described in Section 2.4, with the difference that we reported here the proxy objective function described in Section 4.2.

Table 4: Results with/without symmetry breaking

n	**Gecode+SB**	**Gecode–SB**	**Chuffed+SB**	**Chuffed–SB**
2	**1**	**1**	**1**	**1**
4	UNSAT	UNSAT	UNSAT	UNSAT
6	**1**	**1**	**1**	**1**
8	**1**	**1**	**1**	**1**
10	**1**	N/A	**1**	**1**
12	N/A	N/A	**1**	**1**
14	N/A	N/A	**1**	**1**
16	N/A	N/A	**1**	**1**
18	N/A	N/A	13	**1**
20	N/A	N/A	19	N/A
22	N/A	N/A	N/A	N/A

5 Conclusions

The diversity of the programming languages and their limitations and strengths gave us different views on the problem and tools to handle it, letting us develop different models, having diverse variables and therefore also diverse constraints; without limiting ourselves to just develop three implementations of the same model.

Among all models we developed, unless one specific case (n=18 with Constraint Programming), we had a good improvement for each of them when symmetry breaking (SB) constraints were applied. For all three models, the respective SB constraints allowed us to find a solution when the problem was scaled further by increasing n by one (+2) step. Execution times were not reported, but they also had an improvement by adding SB constraints, outputting faster solutions by reducing the search tree.

We initially implemented constraints that fixed the home and away balances to directly force them on the optimal values. The mentioned constraints are

$$\sum_{t=1}^{n} \Big| \, \#\text{home\_games}(t) - \#\text{away\_games}(t) \, \Big| = n.$$

or alternatively

$$\forall\, t \qquad \Big| \, \#\text{home\_games}(t) - \#\text{away\_games}(t) \, \Big| = 1$$

By strictly constraining the objective function, the model was finding a solution (optimal by problem definition) in significant less time; but this would've changed the optimization problem in a satisfaction problem, so we kept them away to let the solver find the optimal solution without external hints.

The **SAT** approach has a clear limitation in its restriction to purely Boolean variables, which significantly constrained how we could define decision variables.

Despite the considerable effort of implementing and testing four different cardinality encodings, the improvements in the final results remained limited for our formulation. The SAT experiments highlighted that, in our case, performance is influenced more by the modeling choices than by symmetry-breaking techniques. While symmetry breaking yielded incremental improvements, reformulating the model produced substantially larger performance gains.

Linear Programming allowed us to set integer variables in the problem, but it would have been trickier to define the constraints, especially when summing up variables. Further issues encountered with MIP were the linear limitations we had defining functions and constraint, bringing us to define new auxiliary variables and reformulate them as linear inequalities.

Constraint programming was for sure the more flexible method, admitting in the formulation every function we needed and different kind of variables; but it's exactly here where we had to think and experiment plenty of different options to find the best combination that could facilitate the constraint propagation of the solver and consequently the reduction of the search tree.

Authenticity and Author Contribution Statement

We declare that the work reported here is our own and properly cites all external ideas and sources. Parts of this project (text/code/experiments) were assisted by AI tools where indicated; we disclose the tools used and the exact sections where AI assistance was applied expect for the part where it was used to improve the quality of the report, such as grammar, sentence formulation and for code debugging.

Author contributions. Luca Anzaldi developed the SAT-based formulations, including both Model 1 and Model 2. He was responsible the configuration and setup of all scripts, Docker environments, and auxiliary tools required to run the experiments. Martino Michelotti developed the complete CP model. Alex-Cristian Cozma developed the complete MIP model. The report was written by all of the students, mostly reporting the part of the project they were responsible of.

References

[1] MiniZinc: The Modeling Language.
`https://www.minizinc.org/`

[2] CSPLib, *Problem 026: Sports Tournament Scheduling (prob026)*.
`https://www.csplib.org/Problems/prob026/` (Accessed: 2026-01-29)

[3] Leonardo de Moura and Nikolaj Bjørner. *Z3: An Efficient SMT Solver*. In *Proceedings of the 14th International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, LNCS 4963, pp. 337–340, Springer, 2008.
`https://github.com/Z3Prover/z3`

[4] Henrique R. Lopes, Thiago A. F. S. Santos, and Tallys H. Y. Lopes. *Python-MIP: A Python Toolbox for Modeling and Solving Mixed-Integer Linear Programs*. SoftwareX, Volume 12, 2020.
`https://www.python-mip.com/`